

INSTITUT NATIONAL DES SCIENCES DE GESTION

BP : 190 Libreville-Gabon / TEL : 01.73.28.45

Site web : www.insg-gabon.com / Adresse mail : contact@insg-gabon.com



Devoir Framework Web

Niveau : Master 1 IG

Travaux Pratique :

Redigé par :

MOGUANGUE Auguste Arcel

Année académie : 2025-2026

Introduction

L'évolution constante des technologies du développement logiciel a profondément transformé la conception des applications informatiques modernes. Les architectures orientées services et les interfaces de programmation applicative (API) occupent désormais une place centrale dans la communication entre systèmes d'information. Les API REST (Representational State Transfer) s'imposent comme standard incontournable grâce à leur simplicité, leur interopérabilité et leur efficacité.

Ce travail pratique, réalisé dans le cadre du module de développement Web, vise la conception d'une API REST complète pour la gestion d'une collection de livres (Book Management API). Il constitue une première approche concrète du développement backend avec Java et Spring Boot, en couvrant les opérations fondamentales : consultation, ajout et suppression d'ouvrages.

Spring Boot est un framework Java qui simplifie la création d'applications grâce à son mécanisme d'Auto Configuration, son serveur embarqué Tomcat et son système d'injection de dépendances — réduisant considérablement la complexité de configuration.

Les compétences mises en pratique incluent : la création d'un projet Maven, la modélisation objet, l'implémentation d'un contrôleur REST, la manipulation des méthodes HTTP (GET, POST, DELETE) et la validation via Postman.

I. Création du projet Spring Boot

La mise en place de l'environnement de développement a été réalisée via Spring Initializr, la plateforme officielle de génération de projets Spring Boot. Cette étape définit la structure initiale et les dépendances du projet.

Paramètres de configuration du projet

Paramètre	Valeur
Type de projet	Maven
Langage	Java 17
Group ID	com.lab
Artifact ID	bookapi
Dépendance principale	Spring Web

Maven assure la gestion automatique des dépendances et du cycle de vie du projet (compilation, tests, packaging). Le projet a ensuite été ouvert dans Visual Studio Code avec les extensions Java Extension Pack et Spring Boot Extension Pack.

II. Conception de la classe modèle Book

La classe Book, placée dans le package com.lab.bookapi.model, matérialise l'entité centrale du système. Elle suit le principe d'encapsulation de la programmation orientée objet : les données sont protégées et accessibles uniquement via des méthodes dédiées.

Attributs de la classe

- ▶ id (Long) : identifiant unique, clé d'accès à chaque ressource
- ▶ title (String) : titre de l'ouvrage
- ▶ author (String) : nom de l'auteur

Méthodes implémentées

- ▶ Constructeur vide — requis par Spring pour la désérialisation JSON
- ▶ Constructeur paramétré — initialisation rapide des objets
- ▶ Getters (getId, getTitle, getAuthor) — accès en lecture
- ▶ Setters (setId, setTitle, setAuthor) — modification contrôlée

Extrait de code — Classe Book

```
package com.lab.bookapi.model;

public class Book {
    private Long id;
    private String title;
    private String author;
```

```
public Book() {}

public Book(Long id, String title, String author) {
    this.id = id;
    this.title = title;
    this.author = author;
}

// Getters et Setters ...
}
```

III. Développement du contrôleur REST

Le contrôleur `BookController`, placé dans `com.lab.bookapi.controller`, constitue le cœur fonctionnel de l'API. Il reçoit les requêtes HTTP, les traite et retourne les réponses au format JSON. L'annotation `@RestController` combine `@Controller` et `@ResponseBody`, indiquant que chaque méthode sérialise directement son résultat.

Endpoints REST implémentés

Méthode HTTP	URL	Annotation	Rôle
GET	/books	@GetMapping	Récupérer tous les livres
GET	/books/{id}	@GetMapping	Récupérer un livre par ID
POST	/books	@PostMapping	Ajouter un nouveau livre
DELETE	/books/{id}	@DeleteMapping	Supprimer un livre par ID

Extrait de code — BookController

```
@RestController
@RequestMapping("/books")
public class BookController {

    private List<Book> books = new ArrayList<>(Arrays.asList(
        new Book(1L, "Clean Code", "Robert C. Martin"),
        new Book(2L, "Spring Boot in Action", "Craig Walls")
    ));

    @GetMapping
    public List<Book> getAllBooks() {
        return books;
    }

    @GetMapping("/{id}")
    public Book getBookById(@PathVariable Long id) {
        return books.stream()
            .filter(b -> b.getId().equals(id))
            .findFirst().orElse(null);
    }

    @PostMapping
    public Book addBook(@RequestBody Book book) {
        books.add(book);
        return book;
    }
}
```

```
@DeleteMapping("/{id}")
public String deleteBook(@PathVariable Long id) {
    books.removeIf(b -> b.getId().equals(id));
    return "Livre supprimé avec succès.";
}
```

IV. Gestion des données en mémoire

Dans ce laboratoire, aucune base de données n'est utilisée. Les livres sont stockés dans une `ArrayList<Book>` initialisée au démarrage de l'application — une approche volontairement simplifiée pour se concentrer sur les mécanismes REST.

⚠ *Limite importante : toutes les données sont perdues à chaque redémarrage de l'application. Cette approche est idéale pour l'apprentissage, mais inadaptée à la production. Une évolution naturelle serait l'intégration de Spring Data JPA avec une base H2, MySQL ou PostgreSQL.*

V. Exécution de l'application

L'application est lancée via la commande Maven Wrapper, qui compile le projet et démarre le serveur Tomcat embarqué (port 8080 par défaut) :

```
.\mvnw spring-boot:run
```

Le message de confirmation dans la console indique que l'application est prête :

```
Started BookapiApplication in 2.345 seconds (JVM running for 2.891)
```

Le serveur écoute sur `http://localhost:8080`. Spring Boot inclut automatiquement Tomcat — aucune installation externe requise.

VI. Validation des services REST

Trois types de requêtes ont été testés afin de valider le bon fonctionnement de chaque endpoint.

Test GET — Consultation des livres

Depuis le navigateur, l'accès à l'adresse `http://localhost:8080/books` retourne la liste complète au format JSON :

```
[
  { "id": 1, "title": "Clean Code", "author": "Robert C. Martin" },
  { "id": 2, "title": "Spring Boot in Action", "author": "Craig Walls" }
]
```

Test POST — Ajout d'un livre

Via Postman, une requête POST est envoyée avec le corps JSON suivant. Spring Boot déserialise automatiquement ce JSON en objet Book grâce à `@RequestBody` :

```
POST http://localhost:8080/books
Content-Type: application/json

{
  "id": 3,
  "title": "Python",
  "author": "Alex"
}
```

Test DELETE — Suppression d'un livre

La requête DELETE sur `/books/1` supprime le livre d'identifiant 1 et retourne un message de confirmation. Le serveur localise l'objet via `@PathVariable`, le retire de la liste et confirme l'opération.

```
DELETE http://localhost:8080/books/1

→ Réponse : "Livre supprimé avec succès."
```

Conclusion

Ce travail pratique a permis de mettre en œuvre les fondamentaux du développement d'une API REST avec Spring Boot : structuration Maven, modélisation objet, contrôleur REST, méthodes HTTP et tests via Postman.

Les annotations Spring Boot (@SpringBootApplication, @RestController, @GetMapping, @PostMapping, @DeleteMapping, @RequestBody, @PathVariable) ont automatisé la majeure partie des traitements, réduisant considérablement la complexité.

Évolutions possibles

- ▶ Persistance des données avec Spring Data JPA + base H2 ou MySQL
- ▶ Sécurisation des endpoints avec Spring Security (JWT, OAuth2)
- ▶ Gestion centralisée des exceptions avec @ControllerAdvice
- ▶ Documentation automatique de l'API via Swagger / OpenAPI 3.0
- ▶ Tests unitaires et d'intégration avec JUnit 5 et Mockito

Ce laboratoire constitue un socle essentiel pour la conception de systèmes distribués professionnels basés sur les architectures microservices et REST.

Réponses aux questions

Question	Réponse
Rôle de @SpringBootApplication ?	Combine @Configuration, @EnableAutoConfiguration et @ComponentScan. Initialise le contexte Spring, détecte les composants et configure automatiquement les services nécessaires.
À quoi sert @RestController ?	Indique que la classe expose des services REST. Toutes ses méthodes retournent directement des données sérialisées en JSON ou XML, sans moteur de rendu HTML.
Différence @GetMapping / @PostMapping ?	@GetMapping gère les requêtes HTTP GET (consultation). @PostMapping gère les requêtes HTTP POST (création de ressources à partir d'un corps JSON).
Rôle de @RequestBody ?	Convertit automatiquement le corps JSON d'une requête HTTP en objet Java. Utilise Jackson (inclus dans Spring Boot) pour la désérialisation.
À quoi sert @PathVariable ?	Extrait une valeur directement depuis l'URL (ex: /books/{id}) pour l'utiliser comme paramètre de méthode. Facilite l'accès aux ressources individuelles.
Pourquoi utiliser List<Book> ?	Stockage ordonné et dynamique de plusieurs objets Book. Joue le rôle de base de données temporaire en mémoire avec des opérations simples (ajout, suppression, parcours).
Différence classe / objet ?	Une classe est un modèle abstrait définissant structure et comportements. Un objet est une instance concrète de cette classe, avec ses propres valeurs d'attributs.